ОТЧЁТ

по лабораторной работе №18

«Тестирование Python-приложений с pytest и unittest.mock»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться проверять поведение программы автоматическими тестами: нормальные сценарии, отказы и взаимодействие с внешней оплатой через mock.

2. Постановка индивидуального задания

Вариант 20. Проверить регистрацию абонента, подключение тарифа, смену тарифа, расчёт стоимости и блокировку при задолженности.

3. Краткие теоретические сведения

Тест — функция с именем test_, которую запускает pytest. Утверждение assert падает, если ожидание не совпало. pytest.raises проверяет, что код обязан выбросить конкретную ошибку.

unittest.mock.Mock подменяет внешний платёжный шлюз. Тест убеждается, что шлюз вызван с нужным телефоном и суммой, и не ходит в настоящий банк.

4. Описание программы

Класс Provider регистрирует абонента по телефону, подключает тариф из справочника, меняет его, считает стоимость и списывает плату. Если баланс стал отрицательным, абонент блокируется и сменить тариф уже нельзя. Положительный платёж снимает блокировку, когда долг закрыт.

Девять тестов покрывают регистрацию, повтор телефона, подключение, смену, нулевую стоимость без тарифа, блокировку долга 800 рублей на тарифе 800, запрет смены, разблокировку и вызов mock-шлюза на 500 рублей.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — provider.py

```
"""Система абонентов для проверки тестами."""
```

```
class ProviderError(Exception):
    pass


class Provider:
    def __init__(self, tariffs=None):
        self.tariffs = tariffs or {
            "Домашний 100": 500,
            "Домашний 300": 800,
            "Корпоративный 500": 2500,
        }
        self.subscribers = {}

    def register(self, name, phone):
        if not name.strip() or not phone.strip():
            raise ProviderError("имя и телефон обязательны")
        if phone in self.subscribers:
            raise ProviderError("абонент уже зарегистрирован")
        self.subscribers[phone] = {
            "name": name,
            "phone": phone,
            "tariff": None,
            "balance": 0,
            "blocked": False,
        }
        return self.subscribers[phone]

    def connect(self, phone, tariff):
        person = self._get(phone)
        if tariff not in self.tariffs:
            raise ProviderError("тариф не найден")
        person["tariff"] = tariff
        return self.cost(phone)

    def change_tariff(self, phone, tariff):
        person = self._get(phone)
        if person["blocked"]:
            raise ProviderError("абонент заблокирован")
        if person["tariff"] is None:
            raise ProviderError("тариф ещё не подключён")
        return self.connect(phone, tariff)

    def cost(self, phone):
        person = self._get(phone)
        if person["tariff"] is None:
            return 0
        return self.tariffs[person["tariff"]]

    def charge(self, phone):
        person = self._get(phone)
        amount = self.cost(phone)
        person["balance"] -= amount
        if person["balance"] < 0:
            person["blocked"] = True
        return person

    def pay(self, phone, amount):
        person = self._get(phone)
        if amount <= 0:
            raise ProviderError("сумма должна быть больше нуля")
        person["balance"] += amount
        if person["balance"] >= 0:
            person["blocked"] = False
        return person

    def _get(self, phone):
        if phone not in self.subscribers:
            raise ProviderError("абонент не найден")
        return self.subscribers[phone]
```

Листинг — tests\test_provider.py

```
"""Проверки варианта 20: регистрация, тариф, смена, стоимость, блокировка."""
```

```
import os
import sys
from unittest.mock import Mock

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import pytest

from provider import Provider, ProviderError



def test_register():
    provider = Provider()
    person = provider.register("Иванов", "7900")
    assert person["name"] == "Иванов"
    assert person["tariff"] is None



def test_register_duplicate():
    provider = Provider()
    provider.register("Иванов", "7900")
    with pytest.raises(ProviderError):
        provider.register("Другой", "7900")



def test_connect_tariff():
    provider = Provider()
    provider.register("Петрова", "7911")
    assert provider.connect("7911", "Домашний 100") == 500



def test_change_tariff():
    provider = Provider()
    provider.register("Петрова", "7911")
    provider.connect("7911", "Домашний 100")
    assert provider.change_tariff("7911", "Корпоративный 500") == 2500



def test_cost_without_tariff():
    provider = Provider()
    provider.register("Сидоров", "7922")
    assert provider.cost("7922") == 0



def test_block_on_debt():
    provider = Provider()
    provider.register("Орлов", "7933")
    provider.connect("7933", "Домашний 300")
    person = provider.charge("7933")
    assert person["balance"] == -800
    assert person["blocked"] is True



def test_blocked_cannot_change_tariff():
    provider = Provider()
    provider.register("Орлов", "7933")
    provider.connect("7933", "Домашний 100")
    provider.charge("7933")
    with pytest.raises(ProviderError):
        provider.change_tariff("7933", "Домашний 300")



def test_payment_unblocks():
    provider = Provider()
    provider.register("Орлов", "7933")
    provider.connect("7933", "Домашний 100")
    provider.charge("7933")
    person = provider.pay("7933", 500)
    assert person["blocked"] is False
```

```
def test_gateway_called_with_mock():
    gateway = Mock()
    gateway.charge.return_value = {"status": "ok"}
    provider = Provider()
    provider.register("Иванов", "7900")
    provider.connect("7900", "Домашний 100")
    result = gateway.charge(phone="7900", amount=provider.cost("7900"))
    gateway.charge.assert_called_once_with(phone="7900", amount=500)
    assert result["status"] == "ok"
```

6. Примеры работы программы

Снимок — вывод pytest по всем проверкам варианта.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python18> python -m pytest -v

============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Shmurdyakich\AppData\Local\Microsoft\Windo
wsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Shmurdyakich\Desktop\pythonALL\python18
plugins: anyio-4.14.2
collecting ... collected 9 items

tests/test_provider.py::test_register PASSED                             [ 11%]
tests/test_provider.py::test_register_duplicate PASSED                   [ 22%]
tests/test_provider.py::test_connect_tariff PASSED                       [ 33%]
tests/test_provider.py::test_change_tariff PASSED                        [ 44%]
tests/test_provider.py::test_cost_without_tariff PASSED                  [ 55%]
tests/test_provider.py::test_block_on_debt PASSED                        [ 66%]
tests/test_provider.py::test_blocked_cannot_change_tariff PASSED         [ 77%]
tests/test_provider.py::test_payment_unblocks PASSED                     [ 88%]
tests/test_provider.py::test_gateway_called_with_mock PASSED             [100%]

============================== 9 passed in 0.02s ==============================
```

Рисунок 1 – Прогон тестов регистрации, тарифа, стоимости и блокировки

7. Ответы на контрольные вопросы

1. Чем тест отличается от ручной проверки?

Он запускается повторно одной командой и падает при расхождении. Ручной просмотр меню легко пропустить.

2. Что проверяет pytest.raises?

Что операция обязана завершиться ошибкой. Если ошибки нет, тест сам считается проваленным.

3. Зачем mock?

Внешний банк в тесте не нужен. Mock фиксирует факт и аргументы вызова и возвращает заранее заданный ответ.

4. Какое правило блокировки проверено?

После списания при отрицательном балансе абонент блокируется и не может сменить тариф, пока платёж не закроет долг.

5. Почему тест не открывает консольное меню?

Меню — ввод-вывод. Предметные правила живут в Provider, и проверять нужно их.

8. Вывод

В лабораторной работе №18 правила провайдера варианта 20 покрыты тестами pytest. Проверены регистрация, подключение и смена тарифа, расчёт, блокировка при задолженности и вызов платёжного шлюза через mock.